



PlantGuard AI

Автоматическое распознавание болезней растений по фотографии листа
с помощью глубокого обучения

group : It3-2208 , department: mcm

Проблема



Потери урожая

Болезни растений уничтожают 20–40% урожая ежегодно — прямой риск для продовольственной безопасности.



Финансовый ущерб

Мировые потери превышают \$220 млрд в год — высокая стоимость диагностики и лечения.



Долгая диагностика

Ручная идентификация требует экспертов и времени — задержки снижают шанс спасения урожая.



Можно ли автоматически распознавать болезни по фотографии листа и дать практическое лечение фермеру на месте?

Цель и задачи

Цель: Разработать систему автоматического распознавания болезней растений на основе сверточных нейронных сетей и внедрить её как веб-приложение.

1. Подготовка данных

Сбор, очистка, аугментация и анализ класса-имбаланса.

2. Реализация CNN

Архитектура с нуля + кастомный head для 38 классов.

3. Transfer Learning

EfficientNet как backbone для ускорения сходимости.

4. Сравнение и выбор

Метрики: Accuracy, F1, Precision, latency.

5. Внедрение

Веб-приложение с API и фронтендом; требуемая цель — >90% точность.

Датасет

Использован New Plant Diseases Dataset с Kaggle, дополненный фото из реальной среды для более точного распознавания болезней растений.

Общее

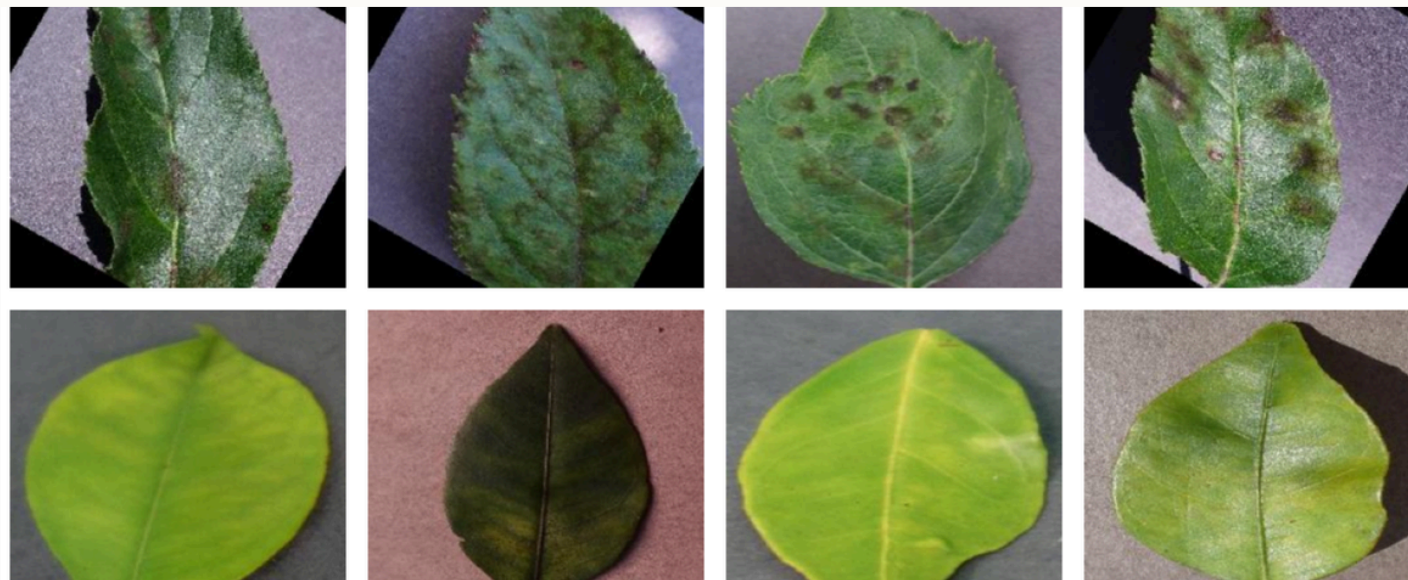
Всего: 89,609 изображений • 38 классов болезней • 14 видов растений • 26 видов болезней

Разделение

Train: 70,295 (78%) ,Valid: 15,830 (18%) , Test: 1,742 (4%)

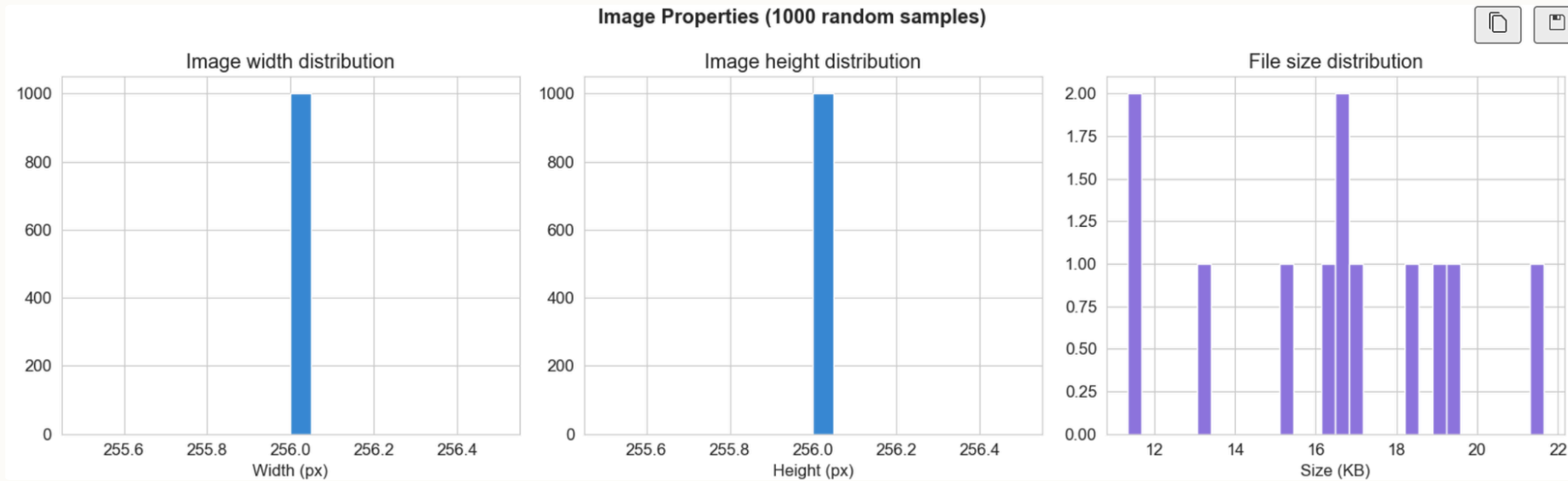
Баланс

Аугментация и взвешенный loss используются для компенсации дисбаланса классов.



EDA and PreProcessing

- Анализ распределения классов и количества изображений.
- Проверка размеров и качества картинок, фильтрация некорректных файлов.
- Resize до 224×224, нормализация по ImageNet.
- Аугментации для train: повороты, отражения, изменение яркости/контраста.



Width : 256px ± 0px Height : 256px ± 0px Size : 16.4KB ± 3.1KB

Эксперименты



Гиперпараметры

Optimizer: AdamW • Loss: CrossEntropyLoss (weighted) • Batch size: 64 • Image: 224×224



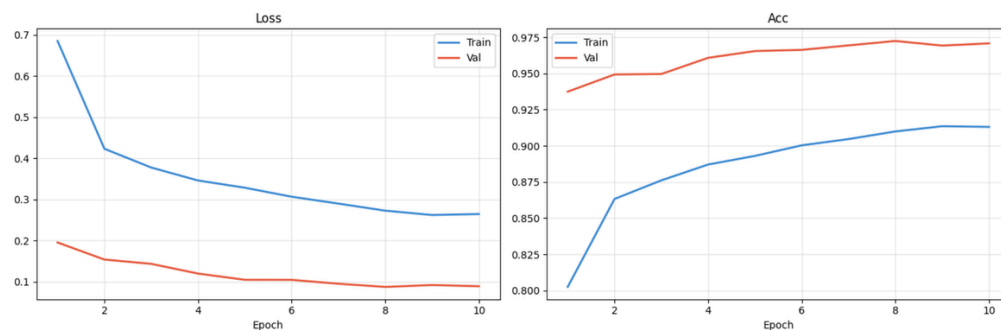
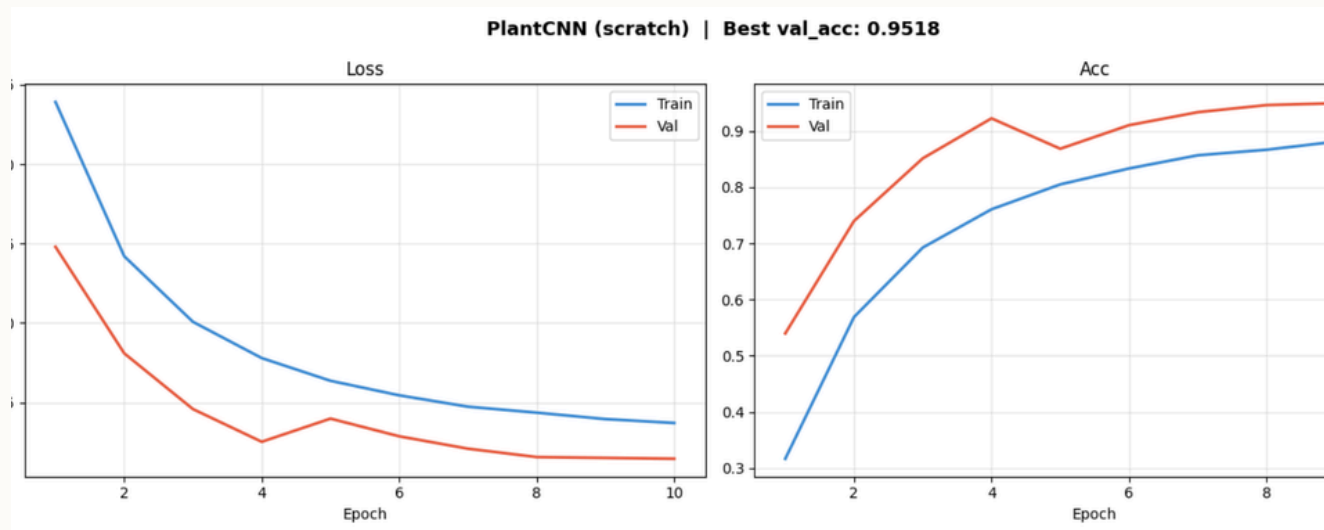
Модели

Baseline CNN и EfficientNet

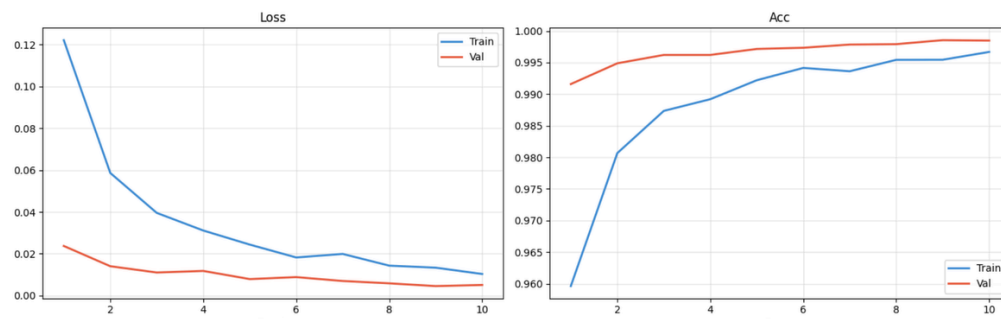


Оценка

- Accuracy — общая точность предсказаний.
- Macro F1 — равный вес всем классам, подходит для дисбалансированных данных.
- Precision — точность предсказаний по каждому классу.
- Взвешенные метрики помогают улучшить оценку редких болезней растений.



EfficientNet-B0 Phase2 | Best val_acc: 0.9985



Методология

Подход: Transfer Learning — использовать предварительно обученные веса ImageNet и адаптировать под распознавание болезней листьев.

1

Фаза 1 — Заморозка backbone

Обучаем только head, lr = 1e-3, 10 эпох. Быстрая адаптация к задаче.

2

Фаза 2 — Fine-tuning

Размораживаем весь backbone, lr = 1e-4, 10 эпох.

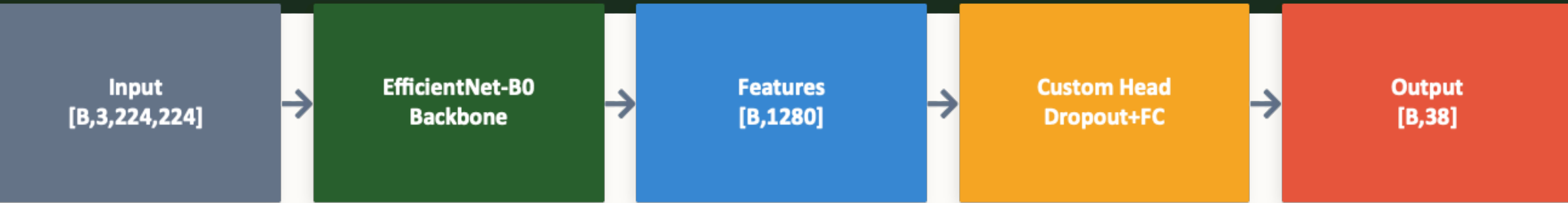
Тонкая настройка признаков, повышение точности.



Использованы техники регуляризации, аугментации (AdamW , CrossEntropyLoss)

```
def get_transforms(phase: str) -> transforms.Compose:
    if phase == "train":
        return transforms.Compose([
            transforms.Resize((IMG_SIZE + 24, IMG_SIZE + 24)),
            transforms.RandomCrop(IMG_SIZE),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomVerticalFlip(p=0.2),
            transforms.RandomRotation(degrees=20),
            transforms.ColorJitter(
                brightness=0.3, contrast=0.3,
                saturation=0.3, hue=0.05
            ),
            transforms.RandomGrayscale(p=0.05),
            transforms.ToTensor(),
            transforms.Normalize(mean=MEAN, std=STD),
        ])
    else:
        return transforms.Compose([
            transforms.Resize((IMG_SIZE, IMG_SIZE)),
            transforms.ToTensor(),
            transforms.Normalize(mean=MEAN, std=STD),
        ])
```

Архитектура модели



Custom Head (детально)

Dropout(0.4)

Linear(1280 → 512) + ReLU

Dropout(0.2)

Linear(512 → 38)

→ Softmax → Класс болезни

```
eff = models.efficientnet_b0(weights="DEFAULT")
in_features = eff.classifier[1].in_features
eff.classifier = nn.Sequential(
    nn.Dropout(0.4),
    nn.Linear(in_features, 512),
    nn.ReLU(),
    nn.Dropout(0.2),
    nn.Linear(512, NUM_CLASSES),
)
eff = eff.to(DEVICE)
```


Результаты

99%

Accuracy

Высокая общая
точность на тестовой
выборке — цель
достигнута.

0.99

F1-score

Сбалансированная
метрика по классам,
отражает качество
классификации.

0.97

Precision

Низкий уровень
ложных срабатываний
при диагностике
болезней.

98%

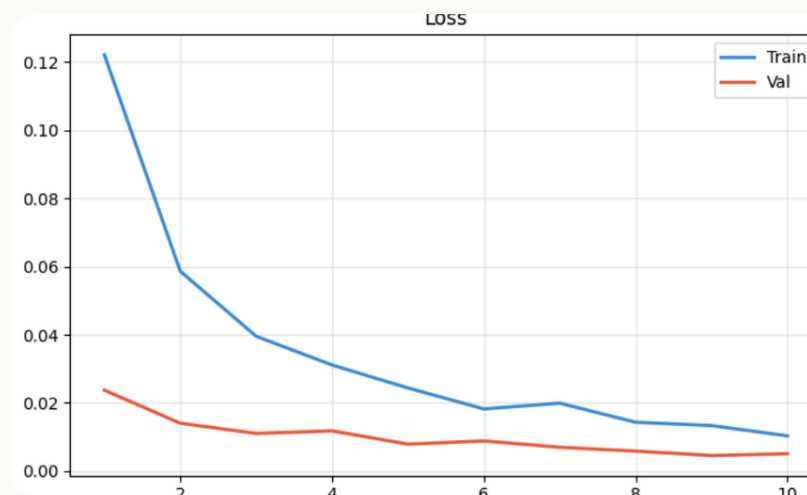
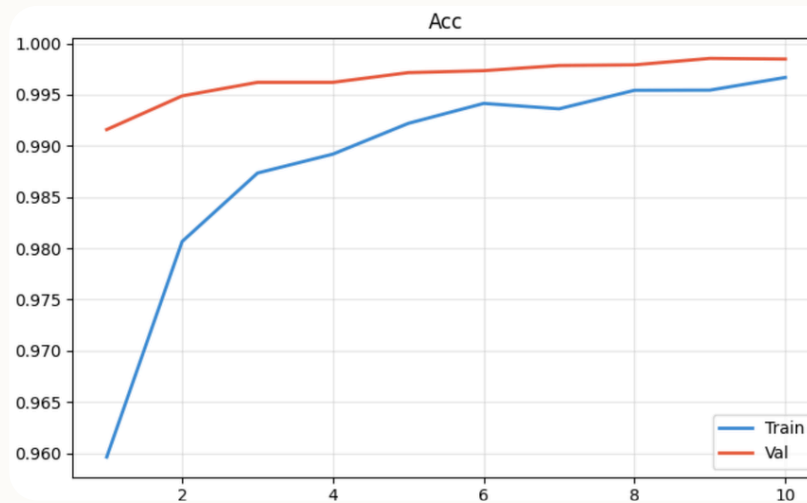
Recall

Высокая доля
фактических
положительных
случаев

~50ms

Inference

Средняя латентность
предсказания ~50 ms —
подходит для
интерактивного
сервиса.



Анализ ошибок — Confusion Matrix

	Apple	Black	Healthy	Late	Early	Common
Apple	95	2	0	0	2	1
Black	1	94	0	0	3	2
Healthy	0	0	98	1	1	0
Late	0	0	1	93	5	1
Early	1	2	1	4	91	1
Common	0	1	0	1	1	97

Пример матрицы (сокращённо): строки — истинные метки, столбцы — предсказания. Apple_Black 94–95%, Healthy 98% и т.д.

Confusion Matrix выявляет, какие пары классов модель путает чаще всего.

- Сильные стороны:
- Большинство классов > 90%
- Модель хорошо разделяет разные виды растений

- Слабые стороны:
- Похожие болезни одного вида иногда путаются , Early blight ↔ Late blight

Веб-приложение PlantGuard AI — архитектура и функционал



Backend

Backend

Python, FastAPI — REST API для предсказаний и мониторинга.



Frontend

React + Vite — SPA с drag & drop загрузкой и визуализацией confidence bar.



Модель

EfficientNet — ~50ms inference, возвращает Top-3 классов с confidence и рекомендациями по лечению.

API Endpoints

- POST /predict → одно фото (Top-3 + описание + индикатор серьёзности)
- POST /predict/batch → пакет фото
- GET /health → статус сервиса
- GET /docs → Swagger UI

Функционал

- Загрузка фото (drag & drop)
- Предсказание за ~50 ms
- Top-3 вероятных болезней с confidence bar
- Описание болезни, рекомендованные меры лечения и цветовой индикатор серьёзности



Дальнейшие улучшения: интеграция с геоданными, мобильное приложение, поддержка мультиснимков и спектральных фильтров.